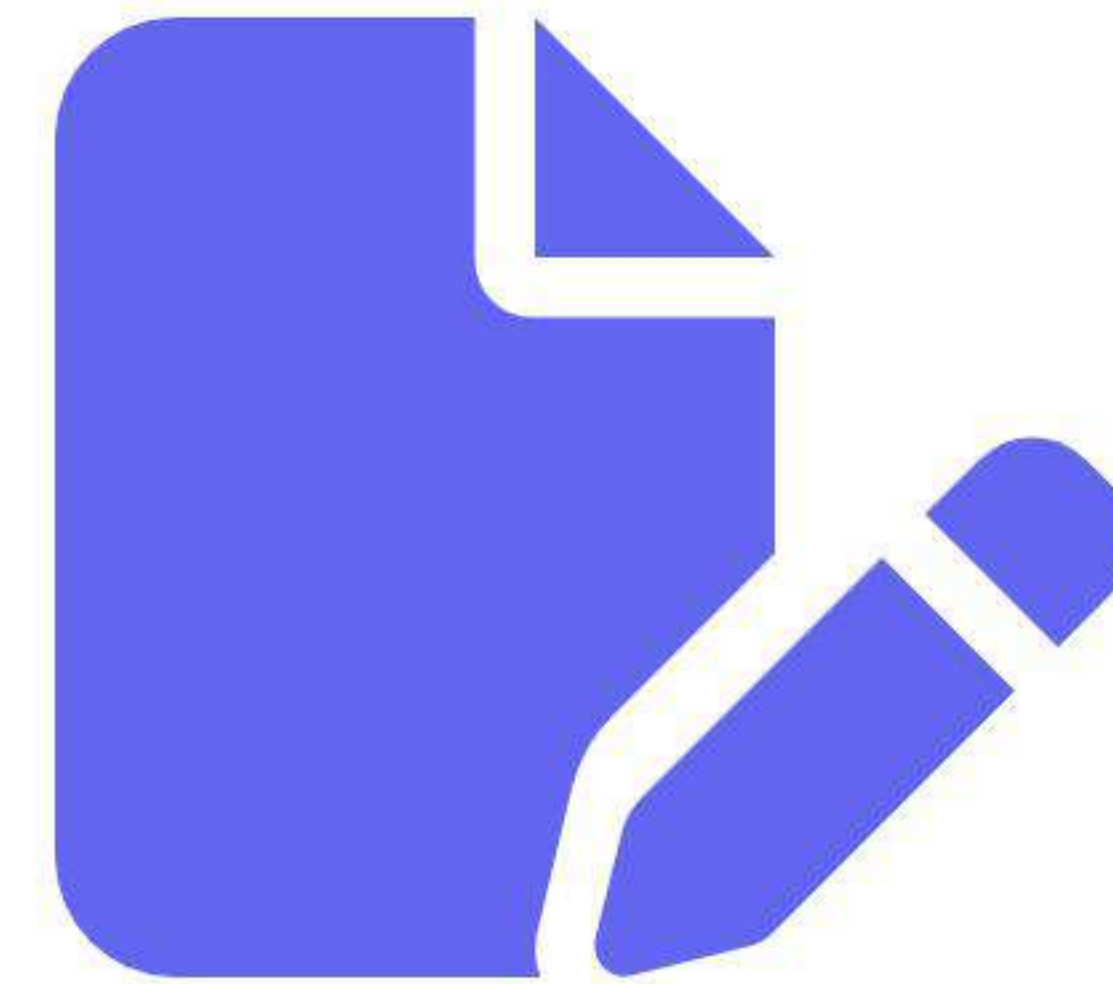


PYTHON TUTORIAL

Writing Data to a File Correctly

Mastering file handling constraints and avoiding the "Trailing Newline" trap.



The Problem

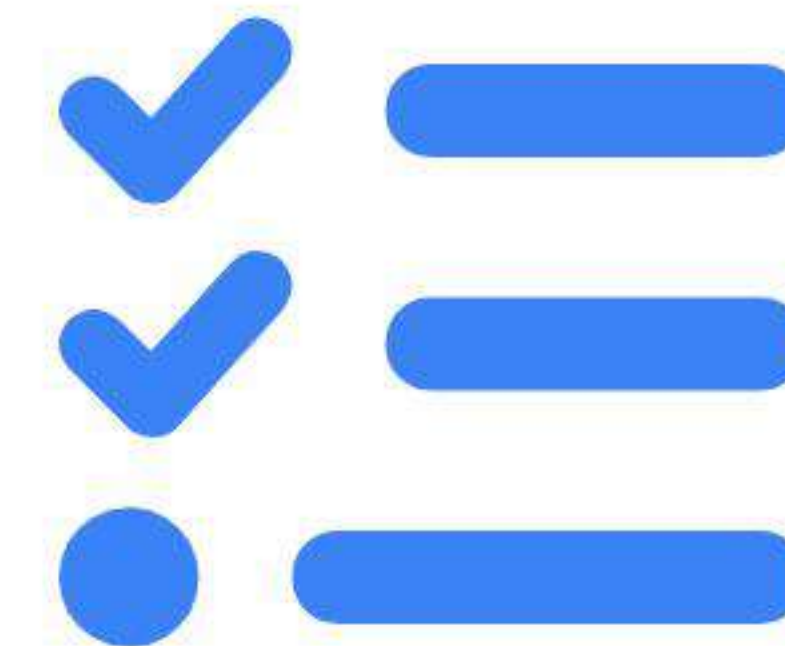
The Input

We have a list of strings:

```
words = ['one', 'two', 'three', 'four',  
         'five']
```

The Goal

- ✓ Write each word to `writing.txt`.
- ✓ Each word must be on a **separate line**.
- ✓ **Constraint:** No extra empty line at the end of the file.



Expected Output vs. Common Mistake

✓ CORRECT OUTPUT

one two three four five EOF

Cursor stops exactly at the end of "five".

× INCORRECT OUTPUT

one two three four five EOF

Extra empty line created by a final `\n`.

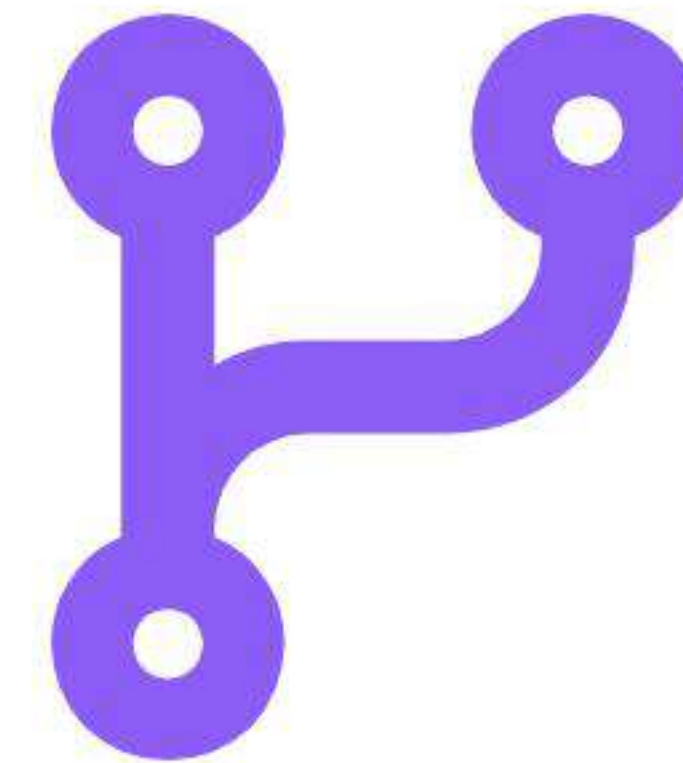
Solution 1: The Conditional Loop

The Strategy

We iterate through the list using an index. For every item, we write the word.

The Trick: We only write a newline character `\n` IF the current word is **not** the last word in the list.

- ✓ Word 1 ("one") → Write "one" + "\n"
- ✓ ...
- ✓ Last Word ("five") → Write "five" (No "\n")



Flowchart: Conditional Logic



If "Is Last?" is TRUE, skip "Write \n".

Code: Conditional Loop

Code Breakdown

```
def write_file_loop(words):  
    with open('writing.txt', 'w') as f:  
        # Enumerate gives us index (i) and value (word)  
        for i, word in enumerate(words):  
  
            # 1. Write the word  
            f.write(word)  
  
            # 2. Add newline ONLY if it's not the last item  
            if i < len(words) - 1:  
                f.write('\n')
```

- `enumerate(words)`:
Allows us to track our position `i` in the list.
- `len(words) - 1`: The index of the very last item.
- **Control**: Gives us precise control over formatting for every single line.

Solution 2: The "Join" Method

The Pythonic Way

Instead of writing to the file repeatedly in a loop, we can prepare the entire text content in memory first.

The `join()` string method creates a single string from a list, placing a separator (`\n`) *between* items, but never at the end.

```
"one" + "\n" + "two" ... + "five"
```



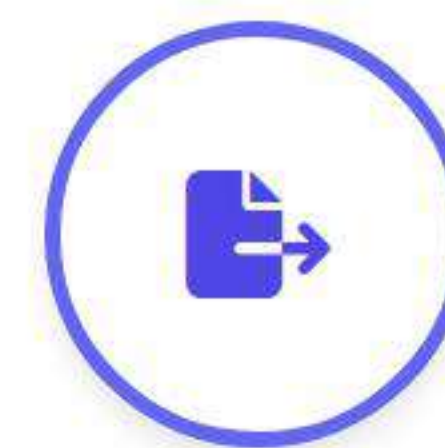
Flowchart: The Join Approach



Input List



Join with \n



Write Once

Code: Join Method

```
def write_file_join(words):  
    with open('writing.txt', 'w') as f:  
  
        # Create one single string with separators  
        # No separator is added after the last item  
        content = '\n'.join(words)  
  
        # Write the entire block at once  
        f.write(content)
```

Code Breakdown

- `'\n'.join(words)`: This performs the magic. It essentially says "Glue these words together using a newline".
- `f.write(content)`: A single I/O operation to save the data.
- **Concise**: Reduces 5+ lines of loop logic into 2 lines.

Comparing Approaches

Feature	Solution 1 (Conditional Loop)	Solution 2 (Join Method)
Complexity	High (Requires index tracking)	Low (Built-in method)
Memory Usage	Low (Processes 1 word at a time)	High (Creates full string in RAM)
Readability	Verbose	Pythonic & Clean
Best For...	Massive lists (millions of items)	Standard lists (typical use)

Summary

To avoid the "trailing newline" trap:

- ✓ Use `'\n'.join(list)` for clean, readable code on standard datasets.
- ✓ Use a **loop with a condition** if you are writing massive amounts of data and need to save memory.

